Quote API
DevOps Project

Component 2

Ansible and Kubernetes Deployment

Submitted by

Sourav Kumar Sahu

USN: 1MS24CY063

DevOps

Course: 25CIAEC59 / 25CYAEC59

Repository

https://github.com/printf-sourav/DevOps-project

Cover and certificate: replace with the prescribed institutional format.

Certificate page - draft for faculty review

Project: Quote API DevOps Project, Component 2.

Student: Sourav Kumar Sahu. USN: 1MS24CY063.

This page is reserved for the institution's prescribed certificate. The assignment refers to a separate title-page and certificate template, but that template was not included in the supplied five-page document. Replace this page and the cover with the official format before submission. This draft does not represent faculty approval or certification.

Institution / Department: _____

Course coordinator: _____

Faculty signature: _____

Date: _____

Abstract

This component deploys the same Quote API built in Component 1. An Ansible role prepares an Ubuntu host, installs Java, copies the executable JAR and starts a systemd service. Docker packages that JAR with a Java runtime. A local Kind Kubernetes cluster runs three replicas behind a Service. The demonstration scales the deployment to five replicas and back to three. Startup, readiness and liveness probes use the API's health route as the deployment enhancement.

Introduction

A successful build is useful only when the application can also be installed and run consistently. Ansible records the server setup as repeatable tasks. Docker provides a portable runtime package. Kubernetes manages the running containers, routes traffic and changes the number of replicas. These steps extend the same application's pipeline rather than introducing a separate application.

Implementation

Development environment

The application targets Java 17 and uses the Maven-built `target/quote-api.jar`. The server demonstration uses Ubuntu under WSL2 with systemd and Ansible Core. The container and cluster demonstration uses a Linux Docker Engine, Kind 0.31.0 and Kubernetes 1.35.0. Docker Desktop was unavailable during setup, so the working Docker engine runs inside Ubuntu WSL. This is still a fully local environment with no paid infrastructure.

Ansible code and configuration

`ansible/inventory` contains one host, localhost, with a local connection. `playbook.yml` applies the `quote_api` role with privilege escalation. The role was initialized with `ansible-galaxy role init quote_api`; unused scaffold directories were omitted from the finished source.

The role's defaults set `app_port` to 8081, an installation directory under `/opt`, a service account, and the path to the built JAR. Its tasks install `openjdk-17-jre-headless`, create an unprivileged account and directory, copy the artifact, and render a systemd unit. Changes to the JAR or unit notify the restart handler. The role enables the service and retries the health request until it returns UP.

The service template passes `APP_PORT`, starts `/usr/bin/java -jar`, restarts on failure and applies basic systemd restrictions. Port 8081 lets this service coexist with the local application and Jenkins. Repeating the playbook without changes should leave the machine unchanged.

Docker image

The Dockerfile starts from `eclipse-temurin:17-jre-jammy`, copies only the packaged JAR, and runs it as numeric user 10001. The Java process listens on container port 8080. The Docker build context excludes all unrelated files through `.dockerignore`. Maven is not installed in the runtime image.

The image is built with `docker build -t quote-api:1.0 .` and tested locally before loading it into Kind. The same artifact is used for the Linux service and the container.

Kubernetes code and deployment

`k8s/deployment.yaml` starts three replicas and matches pods using `app: quote-api`. Each pod runs `quote-api:1.0` with a non-root security context and CPU/memory requests and limits. `k8s/service.yaml` creates a ClusterIP Service whose selector matches those labels. Its port 8080 targets the container port named `http`.

`scripts/deploy.sh` loads the local image into the dedicated Kind cluster, applies both manifests, waits for rollout and lists the pods and Service. The explicit `kind-quote-demo` context keeps these commands scoped to the demo cluster. A service port-forward exposes the API on localhost:8083 for HTTP checks.

Scaling demonstration

`scripts/scale-demo.sh` sets the replica count to three, then five, then three. At each step it waits for rollout and the expected replica count before listing pods. The recorded output shows the number and health of the running replicas. This is manual scaling; an autoscaler is not part of the implementation.

Chosen enhancement: health probes

The Deployment includes readiness and liveness probes that request `/health`. Readiness keeps a pod out of Service traffic until it can answer. Liveness lets Kubernetes restart a container that stops answering. A startup probe gives a newly launched JVM time to initialize before liveness checks begin.

This enhancement was chosen because it uses an existing small endpoint, improves the running deployment and can be inspected directly in the live pod specification. The JSON health route is an HTTP status check; it is not a Prometheus metrics exposition endpoint.

Results

The Ansible run installed and started the service with zero failed tasks. A second unchanged run reported `changed=0` and `failed=0`. The Docker image built successfully and returned UP plus a valid quote response. Kubernetes ran three healthy pods, scaled to five healthy pods, and returned to three. Requests through the Kubernetes Service returned the health response and quote list.

Command-output figures below are rendered transcripts of actual runs. Their source text is included in the evidence folder so the screenshots can be checked against the recorded results.

Ansible initial run and unchanged second run

Ansible service setup and repeat run

Quote API | Sourav Kumar Sahu | 1MS24CY063
Recorded command output - 7 November 2026

```
PLAY [Configure the Quote API server] ************************************
TASK [Gathering Facts] ***************************************************
TASK [quote_api : Install the Java runtime] ******************************
TASK [quote_api : Create a service account] ******************************
TASK [quote_api : Create the application directory] **********************
TASK [quote_api : Copy the built application] ****************************
changed: [localhost]
TASK [quote_api : Install the service definition] ************************
TASK [quote_api : Apply pending service changes] *************************
RUNNING HANDLER [quote_api : Restart quote API] **************************
changed: [localhost]
TASK [quote_api : Enable and start the service] **************************
TASK [quote_api : Wait for the API to become healthy] ********************
PLAY RECAP ***************************************************************
localhost                  : ok=9    changed=2    unreachable=0    failed=0    skipped=0
rescued=0    ignored=0
PLAY [Configure the Quote API server] ************************************
TASK [Gathering Facts] ***************************************************
TASK [quote_api : Install the Java runtime] ******************************
TASK [quote_api : Create a service account] ******************************
TASK [quote_api : Create the application directory] **********************
TASK [quote_api : Copy the built application] ****************************
TASK [quote_api : Install the service definition] ************************
TASK [quote_api : Apply pending service changes] *************************
TASK [quote_api : Enable and start the service] **************************
TASK [quote_api : Wait for the API to become healthy] ********************
PLAY RECAP ***************************************************************
localhost                  : ok=8    changed=0    unreachable=0    failed=0    skipped=0
rescued=0    ignored=0
```

Docker image build and HTTP response

Docker image build and HTTP check

Quote API | Sourav Kumar Sahu | 1MS24CY063
Recorded command output - 7 November 2026

```
Step 1/6 : FROM eclipse-temurin:17-jre-jammy
Step 2/6 : WORKDIR /app
Step 3/6 : COPY --chown=10001:10001 target/quote-api.jar app.jar
Step 4/6 : USER 10001:10001
Step 5/6 : EXPOSE 8080
Step 6/6 : ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
Successfully built 8d04ef2ce69a
Successfully tagged quote-api:1.0
{"status":"UP"}{"status":"UP"}{"id":1,"text":"Start small. Keep improving."}
```

Kubernetes scaling from three to five and back

Kubernetes: three to five to three replicas

Quote API | Sourav Kumar Sahu | 1MS24CY063
Recorded command output - 7 November 2026

```
Scaling quote-api to 3 replicas
deployment.apps/quote-api scaled
deployment "quote-api" successfully rolled out
deployment.apps/quote-api condition met
NAME                         READY   STATUS    RESTARTS   AGE
quote-api-5f98586967-dwz96   1/1     Running   0          37s
quote-api-5f98586967-hlg29   1/1     Running   0          21s
quote-api-5f98586967-jlnck   1/1     Running   0          53s
Scaling quote-api to 5 replicas
deployment.apps/quote-api scaled
Waiting for deployment "quote-api" rollout to finish: 3 of 5 updated replicas are
available...
Waiting for deployment "quote-api" rollout to finish: 4 of 5 updated replicas are
available...
deployment "quote-api" successfully rolled out
deployment.apps/quote-api condition met
NAME                         READY   STATUS    RESTARTS   AGE
quote-api-5f98586967-cd7wz   1/1     Running   0          29s
quote-api-5f98586967-dwz96   1/1     Running   0          67s
quote-api-5f98586967-gdgsp   1/1     Running   0          29s
quote-api-5f98586967-hlg29   1/1     Running   0          51s
quote-api-5f98586967-jlnck   1/1     Running   0          83s
Scaling quote-api to 3 replicas
deployment.apps/quote-api scaled
deployment "quote-api" successfully rolled out
deployment.apps/quote-api condition met
NAME                         READY   STATUS    RESTARTS   AGE
quote-api-5f98586967-dwz96   1/1     Running   0          70s
quote-api-5f98586967-hlg29   1/1     Running   0          54s
quote-api-5f98586967-jlnck   1/1     Running   0          86s
NAME        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
quote-api   ClusterIP   10.96.203.204   <none>        8080/TCP   4m1s
```

Service responses and live health probes

Quote API | Sourav Kumar Sahu | 1MS24CY063
Recorded command output - 7 November 2026

```
GET /health through the Kubernetes Service
{"status":"UP"}
GET /quotes through the Kubernetes Service
[{"id":1,"text":"Start small. Keep improving."},{"id":2,"text":"Build it, test it, then
ship it."},{"id":3,"text":"A little progress every day adds up."}]
Health probes configured in the Deployment
{"failureThreshold":30,"httpGet":
{"path":"/health","port":"http","scheme":"HTTP"},"periodSeconds":5,"successThreshold":1,"t
imeoutSeconds":1}
{"failureThreshold":3,"httpGet":
{"path":"/health","port":"http","scheme":"HTTP"},"periodSeconds":5,"successThreshold":1,"t
imeoutSeconds":1}
{"failureThreshold":3,"httpGet":
{"path":"/health","port":"http","scheme":"HTTP"},"periodSeconds":10,"successThreshold":1,"
timeoutSeconds":1}
```

Conclusion

The same tested JAR was installed as a Linux service and packaged for a local Kubernetes cluster. The Ansible role provides repeatable configuration, the Service provides a stable route, and the scaling demonstration changes capacity without changing application code. Health probes provide a small, practical deployment enhancement.

The official cover/certificate and the course coordinator's GitHub invitation still need the institution's template and confirmed username. The source, commands and recorded deployment evidence are included in the repository.

Resume line

Configured a Java API service with an Ansible role and handlers, then containerized and deployed it on Kubernetes with health probes and manual scaling from three to five replicas.

References

- Assignment brief: DevOps, course 25CIAEC59 / 25CYAEC59, Components 1 and 2.
- Ansible roles: https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_reuse_roles.html
- Ansible systemd service module: https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/systemd_service_module.html
- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Kind quick start: https://kind.sigs.k8s.io/docs/user/quick-start/
- Kubernetes Deployments: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Kubernetes probes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
- Source repository: https://github.com/printf-sourav/DevOps-project